



시험에 나오는것만 공부한다!

시나공시리즈

기출문제

2025년 3회 정보처리기사 실기



정보처리기사 실기 시험은 한국산업인력공단에서 문제를 공개하지 않아 문제 복원에 많은 어려움이 있습니다. 다음에 제시된 문제는 시험을 치른 학생들의 기억을 토대로 복원한 것이므로, 일부 내용이나 문제별 배점이 실제 시험과 다를 수 있음을 알립니다.

저작권 안내

이 자료는 시나공 카페 회원을 대상으로 하는 자료로서 개인적인 용도로만 사용할 수 있습니다. 허락 없이 복제하거나 다른 매체에 옮겨 실을 수 없으며, 상업적 용도로 사용할 수 없습니다.

*** 수험자 유의사항 ***

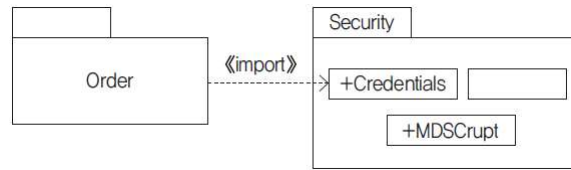
1. 시험 문제지를 받는 즉시 응시하고자 하는 종목의 문제지가 맞는지를 확인하여야 합니다.
2. 시험 문제지 총면수·문제번호 순서·인쇄상태 등을 확인하고, 수험번호 및 성명을 답안지에 기재하여야 합니다.
3. 문제 및 답안(지), 채점기준은 일절 공개하지 않으며 자신이 작성한 답안, 문제 내용 등을 수험표 등에 이기(옮겨 적는 행위) 등은 관련 법 등에 의거 불이익 조치 될 수 있으니 유의하시기 바랍니다.
4. 수험자 인적사항 및 답안작성(계산식 포함)은 흑색 기구만 사용하되, 동일한 한 가지 색의 필기구만 사용하여야 하며 흑색을 제외한 유색 필기구 또는 연필류를 사용하거나 2가지 이상의 색을 혼합 사용하였을 경우 그 문항은 0점 처리됩니다.
5. 답란(답안 기재란)에는 문제와 관련 없는 불필요한 낙서나 특이한 기록사항 등을 기재하여서는 안되며 부정의 목적으로 특이한 표식을 하였다고 판단될 경우에는 모든 문항이 0점 처리됩니다.
6. 답안을 정정할 때에는 반드시 정정부분을 두 줄(=)로 그어 표시하여야 하며, 두 줄로 굿지 않은 답안은 정정하지 않은 것으로 간주합니다.
7. 답안의 한글 또는 영문의 오타자는 오답으로 처리됩니다. 단, 답안에서 영문의 대·소문자 구분, 띄어쓰기는 여부에 관계 없이 채점합니다.
8. 계산 또는 디버깅 등 계산 연습이 필요한 경우는 <문 제> 아래의 연습란을 사용하시기 바라며, 연습란은 채점대상이 아닙니다.
9. 문제에서 요구한 가지 수(항수) 이상을 답란에 표기한 경우에는 답안기재 순으로 요구한 가지 수(항수)만 채점하고 한 항에 여러 가지를 기재하더라도 한 가지로 보며 그 중 정답과 오답이 함께 기재란에 있을 경우 오답으로 처리됩니다.
10. 한 문제에서 소문제로 파생되는 문제나, 가지수를 요구하는 문제는 대부분의 경우 부분채점을 적용합니다. 그러나 소문제로 파생되는 문제 내에서의 부분 배점은 적용하지 않습니다.
11. 답안은 문제의 마지막에 있는 답란에 작성하여야 합니다.
12. 부정 또는 불공정한 방법(시험문제 내용과 관련된 메모지사용 등)으로 시험을 치른 자는 부정행위자로 처리되어 당해 시험을 중지 또는 무효로 하고, 2년간 국가기술자격검정의 응시자격이 정지됩니다.
13. 시험위원이 시험 중 신분확인을 위하여 신분증과 수험표를 요구할 경우 반드시 제시하여야 합니다.
14. 시험 중에는 통신기기 및 전자기기(휴대용 전화기 등)를 지참하거나 사용할 수 없습니다.
15. 국가기술자격 시험문제는 일부 또는 전부가 저작권법상 보호되는 저작물이고, 저작권자는 한국산업인력공단입니다. 문제의 일부 또는 전부를 무단 복제, 배포, 출판, 전자출판 하는 등 저작권을 침해하는 일체의 행위를 금합니다.

※ 수험자 유의사항 미준수로 인한 채점상의 불이익은 수험자 본인에게 전적으로 책임이 있음

문제 1 UML 다이어그램에 대한 다음 설명에서 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오.



(5점)



() 다이어그램

()은(는) UML 정적 모델링의 하나로, 관련있는 객체들을 하나로 묶어 상위 개념으로 추상화한 것이다. 위의 그림과 같이 유스케이스나 클래스 등의 요소들을 그룹화하여 의존 관계를 표현하며, 대규모 시스템에서 주요 요소 간의 종속성을 파악하는 데 사용한다. 시스템의 구조를 간략하게 표현할 수 있고 의존 관계를 명확하게 파악할 수 있어, 불필요한 의존 관계를 제거하거나 간략화함으로써 시스템의 복잡도를 낮추는 곳에도 사용할 수 있다.

답 :

문제 2 다음 설명에 해당하는 UNIX 명령어를 <보기>에서 찾아 쓰시오. (5점)



- (①) : 현재 작업중인 디렉터리 경로를 화면에 표시한다.
- (②) : 현재 디렉터리의 파일 목록을 표시한다.
- (③) : 디렉터리의 위치를 변경한다.
- (④) : 파일을 복사한다.

<보기>

- | | | | |
|-------|--------|---------|------|
| • ls | • fork | • ps | • cp |
| • cat | • pwd | • chmod | • cd |

답

- ①
- ②
- ③
- ④

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.



문제 3 화이트박스 테스트의 검증 기준에 대한 다음 설명에 해당하는 용어를 <보기>에서 찾아 쓰시오. (5점)

테스트 케이스를 소스 코드의 조건문에 포함된 개별 조건식의 결과가 True인 경우와 False인 경우가 한번 이상 수행되도록 설계한다.

<보기>

- 문장 커버리지
- 분기 커버리지
- 조건 커버리지
- 분기/조건 커버리지

답 :



문제 4 다음 설명에서 괄호(①~⑤)에 들어갈 알맞은 용어를 <보기>에서 찾아 기호(㉠~㉤)로 쓰시오. (5점)

- (①) 코드는 데이터 전송 시 오류를 검출하고 수정하는 코드로, 데이터 전송 과정에서 발생한 오류를 검출해 재전송 요구 없이 수신측에서 스스로 수정하는 (②) 방식에 속한다.
- 이에 반해 오류 발생 시 송신측에 재전송을 요구하는 방식은 (③)이며, 대표적인 종류에는 (④) 검사와 (⑤) 검사가 있다.
- (①) 코드 : 전송 데이터에 n비트의 검사 비트를 추가해 오류를 검출하고, 수정한다.
- (④) 검사 : 전송 데이터의 한 블록 끝에 1비트의 검사 비트 추가하여 오류를 검출한다.
- (⑤) 검사 : 송신측과 수신측이 동일한 특정 다항식을 사용하여 오류를 검출한다.

<보기>

- | | | | |
|----------|-------|-------|-----------|
| ㉠ CRC | ㉡ FEC | ㉢ BEC | ㉣ NAK |
| ㉤ Parity | ㉥ MD5 | ㉦ BCD | ㉧ Hamming |

답

- ①
- ②
- ③
- ④
- ⑤

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.



문제 5 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
#include <stdio.h>
struct Test {
    int i;
    const char *g;
};
int main( ) {
    struct Test test[ ] = {{1, "AB"}, {2, "DC"}, {3, "EB"}};
    struct Test *p = &test[1];
    printf("%s", p->g + (p->i - 1));
    return 0;
}
```

답 :



문제 6 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
enum Tri {
    A("A"), B("AB"), C("ABC");
    private String code;
    Tri(String code) {
        this.code = code;
    }
    public String code( ) {
        return code;
    }
}

public class Main {
    public static void main(String[] args) {
        Tri t = Tri.values( )[Tri.A.name( ).length( )];
        System.out.print(t.code( ));
    }
}
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.



문제 7 <A> 테이블과 테이블을 참고하여 <SQL문>의 실행 결과를 쓰시오. (5점)

<A>	
NAME	RULE
Smith	S%
Allen	%T%
Scott	

<SQL문>

```
SELECT COUNT(*) CNT FROM A CROSS JOIN B WHERE A.NAME LIKE B.RULE;
```

답 :



문제 8 다음 설명에 해당하는 용어를 <보기>에서 찾아 쓰시오. (5점)

- 인터넷 애플리케이션에서 사용자 인증에 사용되는 표준 인증 방법으로, 공개 API(OpenAPI)로 구현되었다.
- 인터넷 사용자가 웹사이트나 애플리케이션에 비밀번호를 제공하지 않고 자신에게 접근 권한을 부여하여 사용할 수 있다.
- 2010년 ETF에서 1.0이 공식 표준안으로 발표되었다.

<보기>

- | | | | |
|----------|--------|---------|--------|
| • OpenID | • IDEA | • OAuth | • SSPI |
| • SASL | • PEAP | • OIDC | • JAAS |

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하기 바랍니다.

문제 9 다음 설명에 해당하는 알맞은 용어를 영문 약어로 쓰시오. (5점)



- 사용자가 패스워드를 요청할 때마다 암호 알고리즘과 해시 함수를 이용하여 새로운 일회용 패스워드를 생성하며, 한 번 사용된 패스워드는 폐기되어 재사용할 수 없다.
- 주로 은행 및 금융 거래 시 사용자 본인 인증을 강화하기 위해 널리 사용된다.

답 :

문제 10 다음 Java로 구현된 프로그램을 분석하여 괄호에 들어갈 알맞은 코드를 쓰시오. (5점)



```
interface Cals {  
    public void get(int v);  
}  
  
class Test (    ) Cals {  
    public void get(int v) {  
        System.out.print(v*v);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Cals a = new Test();  
        a.get(10);  
    }  
}
```

답 :

※ 다음 여백은 연습란으로 사용하시기 바랍니다.



문제 11 다음 Python으로 구현된 프로그램을 분석하여 출력값(①~⑧)에 들어갈 알맞은 값을 쓰시오. (5점)

```
data = [
    [3, 5, 2, 4, 1],
    [4, 5, 1],
    [4, 4, 1, 5, 4],
    [4, 5]
]
result = { }
for index, lis in enumerate(data):
    list_sum = sum(lis)
    list_len = len(lis)
    result[index] = (list_sum, list_len)
print(result)
```

<출력값>

```
{0: (①, ②), 1: (③, ④), 2: (⑤, ⑥), 3: (⑦, ⑧)}
```

답

- ①
- ②
- ③
- ④
- ⑤
- ⑥
- ⑦
- ⑧

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.



문제 12 다음 <R>과 <S> 테이블에 대해 'R ÷ S'를 실행했을 때 출력되는 결과를 테이블로 작성 하시오. (5점)

<R>		<S>	
A	B	B	
a1	b1	b1	
a2	b2	b3	
a1	b3		

답 :



문제 13 접근 통제(Access Control)에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. (5점)

(①)	- 주체와 객체의 등급을 비교하여 접근 권한을 부여하는 방식이다. - 시스템이 접근통제 권한을 지정한다. - 데이터베이스 객체별로 보안 등급을 부여할 수 있다. - 사용자별로 인가 등급을 부여할 수 있다.
(②)	- 사용자의 역할에 따라 접근 권한을 부여하는 방식이다. - 중앙관리자가 접근통제 권한을 지정한다. - 임의 접근통제와 강제 접근통제의 단점을 보완하였다. - 다중 프로그래밍 환경에 최적화된 방식이다.
(③)	- 데이터에 접근하는 사용자의 신원에 따라 접근 권한을 부여하는 방식이다. - 데이터 소유자가 접근통제 권한을 지정하고 제어한다. - 객체를 생성한 사용자가 생성된 객체에 대한 모든 권한을 부여받고, 부여된 권한을 다른 사용자에게 허가할 수도 있다.

<보기>

- DAC(Discretionary Access Control) - MAC(Mandatory Access Control) - RBAC(Role Based Access Control)

답

- ①
- ②
- ③



문제 14 다음 테스트 케이스를 참조하여 괄호(①~③)에 들어갈 테스트 케이스의 구성 요소를

<보기>에서 찾아 쓰시오. (5점)

식별자_ID	테스트 항목	(①)	(②)	(③)
LS_W10_35	로그인 기능	사용자 초기 화면	아이디(test_a01) 비밀번호(203a1d5ffa1)	로그인 성공
LS_W10_36	로그인 기능	사용자 초기 화면	아이디(test_a01) 비밀번호(1234)	로그인 실패(1) - 비밀번호 비일치
LS_W10_37	로그인 기능	사용자 초기 화면	아이디(" ") 비밀번호(" ")	로그인 실패(2) - 미입력

<보기>

• 요구 절차	• 의존성 여부	• 테스트 데이터	• 테스트 조건
• 하드웨어 환경	• 예상 결과	• 소프트웨어 환경	• 성공/실패 기준

답

- ①
- ②
- ③



문제 15 릴레이션을 구성하는 용어들에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 답을

<보기>에서 찾아 쓰시오. (5점)

• (①) : 릴레이션을 구성하는 각각의 행을 의미하며, 파일 구조에서는 레코드에 해당함
• (②) : 데이터 개체를 구성하고 있는 속성들에 데이터 타입이 정의되어 구체적인 데이터 값을 가진 것으로, 실제 값을 가진 튜플을 의미함
• (③) : 튜플의 개수를 의미함

<보기>

• 도메인	• 차수	• 속성	• 튜플
• 디그리	• 카디널리티	• 릴레이션 스키마	• 릴레이션 인스턴스

답

- ①
- ②
- ③

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.



문제 16 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
#include <stdio.h>
int main(void) {
    char str[ ] = "REPUBLICOFKOREA";
    int a = 0;
    while (str[a] != '\0')
        ++a;
    putchar(str[a-2]);
    return 0;
}
```

답 :



문제 17 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
#include <stdio.h>
int main( ) {
    int x = 7, y = 4, z;
    z = y % 3 < 3 ? 2 : 1;
    z = z & z >> 1;
    z = x > 5 && z <= 3 ? z * x : z / x;
    printf("%d", z);
    return 0;
}
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.



문제 18 다음 Java로 구현된 프로그램을 분석하여 괄호에 들어갈 알맞은 코드를 쓰시오. (5점)

```
class Rectangle {
    int x, y;
    Rectangle(int x, int y) {
        this.x = x;
        this.y = y;
    }
    int getArea( ) {
        return x*y;
    }
}

class Square extends Rectangle {
    Square(int a) {
        (    )(a, a);
    }
    int getSquareArea( ) {
        return x*y;
    }
}

public class Main {
    public static void main(String[] args) {
        Square sq = new Square(10);
        System.out.println(sq.getSquareArea( ));
    }
}
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.



문제 19 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
#include <stdio.h>
struct Node {
    struct Node* next;
    unsigned int x;
};

int main( ) {
    struct Node t1 = {0, 5u};
    struct Node t2 = {0, 7u};
    struct Node t3 = {0, 11u};

    t3.next = &t2;
    t2.next = &t1;

    struct Node* curr = &t3;
    int sum = 0;

    while (curr) {
        sum = sum * 3 + curr->x;
        curr = curr->next;
    }

    sum = (sum ^ 42u) + 100u;
    printf("%u\n", sum);
}
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.



문제 20 다음 <A> 테이블을 참조하여 <SQL문>을 실행했을 때 출력되는 결과를 쓰시오. (<A> 테이블에 표시된 'NULL'은 값이 없음을 의미한다.) (5점)

<A>

COL1	COL2
2	NULL
3	6
2	3
NULL	3
4	5

<SQL문>

```
SELECT COUNT(COL2)
FROM A
WHERE COL1 IN (2, 3)
      OR COL2 IN (3, 5);
```

답 :

※ 다음 여백은 연습란으로 사용하기 바랍니다.

기출문제 정답 및 해설

[문제 1]

패키지

[문제 2]

① pwd ② ls ③ cd ④ cp

[문제 3]

조건 커버리지

[문제 4]

① ○ ② ⊐ ③ ⊑ ④ ⊒ ⑤ ⊓

[문제 5]

C

[해설]

```
#include <stdio.h>
struct Test {
    int i;
    const char *g;
};

// 구조체 Test를 정의합니다.
// 정수형 변수 i를 선언합니다.
// 문자형 포인터 상수 g를 선언합니다.

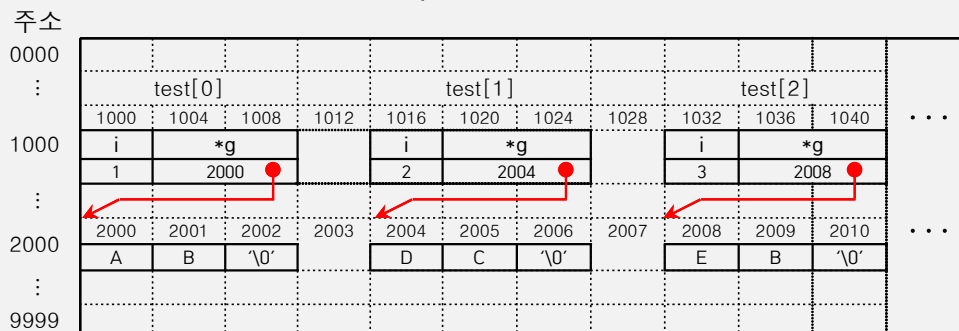
struct Test
{
    int i(4Byte)
    char *g(8Byte)
    정수를 저장할 멤버
    문자열이 저장된 주소를 저장할 멤버
};

int main( ) {
    ❶ struct Test test[ ] = {{1, "AB"}, {2, "DC"}, {3, "EB"}};
    ❷ struct Test *p = &test[1];
    ❸ printf("%s", p->g + (p->i - 1));
    ❹ return 0;
}
```

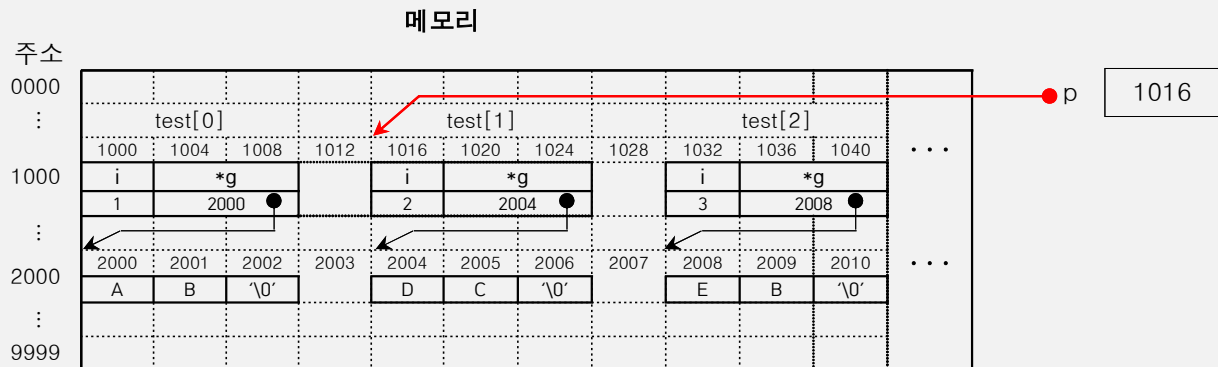
모든 C 프로그램은 반드시 main() 메소드에서 시작한다.

- ❶ 구조체 Test 자료형으로 3개짜리 배열 test를 선언하고 초기화한다. 배열을 선언할 때 배열의 크기를 지정하지 않으면 초기값의 개수로 배열의 요소가 만들어진다.

메모리



- ❷ Test 구조체를 가리키는 포인터 변수 p를 선언하고, test[1]의 주소를 저장한다. p는 1016번지 이후의 12Byte를 의미한다. (이후 그림에서 지정한 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했다.)



- ③ p가 가리키는 곳의 g가 가리키는 곳의 주소에서 1만큼 증가한 곳의 값을 문자열로 출력한다. 문자열의 출력은 문자열의 시작 위치부터 문자열의 끝을 의미하는 널 문자('\0') 전까지를 출력한다.

$p \rightarrow g + (p \rightarrow i - 1)$

①

②

③

· ① $p \rightarrow i$: p가 가리키는 곳의 i 값은 2다.

· ② $① - 1 \rightarrow 2 - 1 : 1$

· ③ $p \rightarrow g + ② \rightarrow p \rightarrow g + 1$: p가 가리키는 곳의 g가 가리키는 곳의 주소인 2004에서 1만큼 증가한 곳인 2005 번지의 값은 "C"다.

결과 **C**

- ④ main() 함수에서의 'return 0'은 프로그램의 종료를 의미한다.

[문제 6]

AB

[해설]

```
enum Tri {
    ① A("A"), B("AB"), C("ABC"); // 열거형 Tri 클래스를 정의한다.
    private String code; // 열거형 Tri 클래스의 객체 상수 A, B, C를 정의한다.
    ② Tri(String code) { // 문자열 변수 code를 객체 상수의 속성으로 정의한다.
        ③ this.code = code; // 생성자로, 열거형 상수가 정의될 때 자동으로 호출된다.
    }
    ③ public String code( ) { // 메소드 code를 객체 상수와 메소드로 정의한다.
        ④ return code; // 해당 객체 상수의 속성 code를 반환한다.
    }
}

public class Main {
    public static void main(String[] args) {
        ① Tri t = Tri.values()[Tri.A.name().length()];
        ②⑤ System.out.print(t.code( ));
    }
}
```

모든 Java 프로그램은 main() 메소드에서 시작하는데, main() 메소드가 시작되기 전 다음과 같이 Tri 열거형 클래스의 객체 상수 초기화 과정이 수행된다.

- ① 열거형(enum) 객체 상수 A를 선언하면서 "A"를 인수로 생성자를 호출한다. 객체 상수가 정의될 때 해당 객체가 문자열 변수 code와 문자열을 반환하는 code() 메소드도 함께 정의된다.
- ② 열거형 Tri의 생성자 Tri()의 시작점이다. ①번에서 전달한 "A"를 code가 받는다.
- ③ code에 "A"를 저장한다. 즉 객체 상수 A의 code에 "A"가 저장된다.

※ 위와 같은 과정으로 객체 상수 B, C의 초기화도 수행되며, 수행된 결과는 다음과 같다. 열거형 상수는 내부적으로 선언 순서에 따라 순서가 부여되며, 이 순서는 배열에서 인덱스로 사용된다.

Tri[0] Tri.A		Tri[1] Tri.B		Tri[2] Tri.C	
code	code()	code	code()	code	code()
"A"		"AB"		"ABC"	

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

❶ Tri 객체 변수 t에 Tri.values()[1], 즉 Tri.B를 저장한다.

Tri.values()[Tri.A.name().length()];

①
②
③

· ① Tri.A.name() : Tri의 상수 A의 이름을 문자열 "A" 형태로 반환함

※ name() : 열거형 상수의 이름을 문자열로 반환함

· ② ①.length() → "A".length() : 문자열 "A"의 길이인 1을 반환함

※ length() : 문자열의 길이를 반환함

· ③ Tri.values()[②] → Tri.values()[1] : 배열로 변환된 Tri의 상수 중 인덱스가 1인 요소, 즉 Tri.B를 반환함

※ values() : 열거형 상수를 배열의 형태로 변환함

❷ t, 즉 Tri.B의 code() 메소드를 호출한 후 돌려받은 값을 출력한다. ❸번으로 이동한다.

❸ 문자열을 반환하는 code() 메소드의 시작점이다. ❷번에서 Tri.B의 code() 메소드가 호출되었으므로 현재 code는 "AB"이다.

❹ code의 값 "AB"를 메소드를 호출했던 ❺번으로 반환한다.

❺ ❹번으로부터 반환받은 값 AB를 출력한다.

결과 AB

[문제 7]

4

[해설]

```
SELECT COUNT(*) CNT
FROM A CROSS JOIN B
WHERE A.NAME LIKE B.RULE;
```

질의문은 각 절을 분리하여 이해하면 쉽습니다.

· SELECT COUNT(*) CNT : 튜플의 개수를 표시하되, 필드명은 'CNT'로 표시합니다.

※ 'SELECT COUNT(*) AS CNT'에서 AS가 생략된 형태입니다.

· FROM A CROSS JOIN B : <A>와 를 교차 조인(CROSS JOIN)한 결과를 대상으로 검색합니다.

A.NAME	B.RULE
Smith	S%
Smith	%T%
Allen	S%
Allen	%T%
Scott	S%
Scott	%T%

· WHERE A.NAME LIKE B.RULE : <A> 테이블의 'NAME' 필드 값이 테이블의 'RULE' 필드에 저장된 문자열 패턴과 일치하는 튜플만을 대상으로 합니다.

※ 테이블의 'RULE' 필드에 저장된 값은 'S%'와 '%T%'와 같이 문자 패턴인 '%' 기호가 포함되어 있으므로, 조건문의 LIKE 연산자와 결합되면 다음과 같이 적용됩니다.

- A.NAME LIKE S% : 'A.NAME'이 "S"로 시작하는 레코드를 검색

NAME	RULE
Smith	S%
Smith	%T%
Allen	S%
Allen	%T%
Scott	S%
Scott	%T%

- A.NAME LIKE %T% : 'A.NAME'이 "T"를 포함하는 레코드를 검색

NAME	RULE
Smith	S%
Smith	%T%
Allen	S%
Allen	%T%
Scott	S%
Scott	%T%

※ CROSS JOIN된 결과에서 조건을 만족하는 튜플은 다음과 같습니다. 그러므로 검색된 튜플의 개수는 4입니다.

NAME	RULE
Smith	S%
Smith	%T%
Scott	S%
Scott	%T%

[문제 8]

oAuth

[문제 9]

OTP

[문제 10]

implements

[해설]

```

① interface Cals {
②     public void get(int v);
}

③ class Test implements Cals {
④     ③ public void get(int v) {
⑤         System.out.print(v*v);
⑥     }
}

public class Main {
    public static void main(String[] args) {
⑦         Cals a = new Test();
⑧         a.get(10);
⑨     } ⑩
}

```

① 인터페이스 Cals를 선언한다.

② 반환값이 없는 get()을 선언한다. 인터페이스에 선언된 메소드는 선언만 있고 내부에 실행 코드가 없는 추상 메소드이므로, 이후 상속 관계가 설정된 자식 클래스에서 재정의한 후 사용한다.

③ Cals 인터페이스를 상속받는 클래스 Test를 정의한다.

④ 반환값이 없는 get()을 정의한다. get() 메소드는 ③에서 선언된 추상 메소드(③)를 재정의하는 것이다.

※ 추상 메소드는 부모 클래스가 자식 클래스에게 주는 의무와 같습니다. 부모 클래스와 상속 관계에 있다면 반드시 부모 클래스의 추상 메소드를 재정의해야 합니다. 그렇지 않으면 프로그램에 오류가 발생합니다.

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

① Test 클래스의 생성자를 이용하여 Cals의 객체 변수 a를 선언한다. 부모 클래스의 객체 변수를 선언하면서 자식 클래스의 생성자를 사용하면 형 변환이 발생한다. 이렇게 형 변환이 발생했을 때 부모 클래스와 자식 클래스에 동일한 속성이나 메소드가 있으면 부모클래스의 속성이나 메소드가 자식 클래스의 속성이나 메소드로 재정의된다.

② 10을 인수로 하여 a의 get() 메소드를 호출한다. ③번으로 이동한다.

※ get() 메소드는 a 객체의 자료형이 Cals이므로 Cals 클래스의 get()이라고 생각할 수 있지만 ①번에서 클래스 형 변환이 발생하였고, get() 메소드가 자식 클래스에서 재정의되었으므로 Test 클래스의 get() 메소드가 수행됩니다.

③ 반환값이 없는 get() 메소드의 시작점이다. ②번에서 전달받은 10을 v가 받는다.

④ v*v의 결과값인 100을 출력한 후 메소드가 종료되면, 호출했던 ②번의 다음 줄인 ⑤번으로 이동하여 프로그램을 종료한다.

결과 100

[문제 11]

① 15 ② 5 ③ 10 ④ 3 ⑤ 18 ⑥ 5 ⑦ 9 ⑧ 2

[해설]

```

❶ data = [
    [3, 5, 2, 4, 1],
    [4, 5, 1],
    [4, 4, 1, 5, 4],
    [4, 5]
]
❷ result = { }
❸ for index, lis in enumerate(data):
❹     list_sum = sum(lis)
❺     list_len = len(lis)
❻     result[index] = (list_sum, list_len)
❼ print(result)

```

❶ 리스트 안에 리스트가 저장되는 다차원 리스트 data를 선언하고, 다음과 같이 초기화한다.

리스트 data

	[0]	[1]	[2]	[3]	[4]
data[0]	3	5	2	4	1
data[1]	4	5	1		
data[2]	4	4	1	5	4
data[3]	4	5			

❷ 딕셔너리 result를 선언한다.

❸ 리스트 data의 요소 수만큼 ❹~❻번 문장을 반복 수행한다. 리스트 data는 4개의 요소를 가지므로 각 요소의 '인덱스'와 '요소값'을 index와 lis에 할당하면서 ❹~❻번 문장을 4회 수행한다.

· **enumerate()** : 반복 가능한 객체를 대상으로 해당 객체의 요소를 '인덱스, 요소값'의 형태로, 인덱스와 요소를 묶어서 반환함

❹ list_sum에 계산된 lis의 모든 요소의 합을 저장한다.

· **sum()** : 리스트의 모든 요소의 합을 계산하여 반환함

❺ list_len에 계산된 lis 요소의 개수를 저장한다.

· **len()** : 리스트 요소의 개수를 반환함

❻ 딕셔너리 result의 index 위치에 (list_sum, list_len)을 값으로 저장한다.

반복문 실행에 따른 변수들의 변화는 다음과 같다.

index	lis	list_sum	list_len	result
0	3 5 2 4 1	15	5	[0] 15 5
1	4 5 1	10	3	[0] [1] 15 5 10 3
2	4 4 1 5 4	18	5	[0] [1] [2] 15 5 10 3 18 5
3	4 5	9	2	[0] [1] [2] [3] 15 5 10 3 18 5 9 2

❼ 딕셔너리 result를 출력한다.

결과 {0: (15, 5), 1: (10, 3), 2: (18, 5), 3: (9, 2)}

[문제 12]

A
a1

[해설]

‘R ÷ S’에서 ÷는 R의 속성 중 S의 속성값을 모두 가진 튜플에서 S가 가진 속성을 제외한 속성만을 구하는 연산입니다.

① R의 속성에서 S의 속성값을 모두 가진 튜플을 추출합니다.

<R>		<S>		<R>	
A	B	B		A	B
a1	b1	b1		a1	b1
a2	b2	b3		a1	b3
a1	b3				

② ①의 결과에서 S가 가진 속성을 제외한 속성만 추출합니다. 결과를 만족하는 <R>의 속성은 ‘A’이며, ‘A’ 속성의 고유한 값은 a1입니다.

<R>		<R>	
A		A	
a1		a1	
a1			

[문제 13]

① MAC ② RBAC ③ DAC

[문제 14]

① 테스트 조건 ② 테스트 데이터 ③ 예상 결과

[문제 15]

① 튜플 ② 릴레이션 인스턴스 ③ 카디널리티

[문제 16]

E

[해설]

```
#include <stdio.h>
int main(void) {
  ❶ char str[ ] = "REPUBLICOFKOREA";
  ❷ int a = 0;
  ❸ while (str[a] != '\0')
  ❹   ++a;
  ❺ putchar(str[a-2]);
  ❻ return 0;
}
```

❶ 문자형 배열 str을 선언하고 초기화한다. 배열의 크기를 생략하면, 초기값으로 지정한 개수만큼 배열이 선언 되는데, 문자열의 경우 문자열의 끝을 의미하는 널 문자('\0')가 추가되어 16개의 요소를 갖는 배열로 선언된다.

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]
str	R	E	P	U	B	L	I	C	O	F	K	O	R	E	A	\0

- ② 정수형 변수 a를 선언하고, 0으로 초기화한다.
 ③ str[a]가 '\0'(널 문자)가 아닌 동안 ④번을 반복 수행한다.
 ④ 'a = a + 1;'과 동일하다. a의 값을 1씩 증가시킨다.
 반복문 실행에 따른 변수들의 변화는 다음과 같다.

a	str[a]	str[a] != '\0'
0	R	No
1	E	No
2	P	No
3	U	No
4	B	No
5	L	No
6	I	No
7	C	No
8	O	No
9	F	No
10	K	No
11	O	No
12	R	No
13	E	No
14	A	No
15	\0	YES

- ⑥ str[a-2], 즉 str[13]의 값 E를 출력한다.
 결과 E XXXXXXXXXX
 ⑦ main() 함수에서의 'return 0;'은 프로그램의 종료를 의미한다.

[문제 17]

0

[해설]

```
#include <stdio.h>
int main( ) {
  ① int x = 7, y = 4, z;
  ② z = y % 3 < 3 ? 2 : 1;
  ③ z = z & z >> 1;
  ④ z = x > 5 && z <= 3 ? z * x : z / x;
  ⑤ printf("%d", z);
  ⑥ return 0;
}
```

- ① 정수형 변수 x, y, z를 선언하고, x는 7로, y는 4로 초기화한다.
 ② y를 3으로 나눈 나머지가 3보다 작으면 z에 2를 저장하고, 아니면 1을 저장한다. y의 값 4를 3으로 나눈 나머지 1은 3보다 작으므로 z에는 2가 저장된다.
 ③ 'z & z >> 1'의 결과 0을 z에 저장한다.
 z & z >> 1
 ①
 ②
 · ① z >> 1 : z의 값 2(0010₂)를 오른쪽으로 1비트 이동시킨 1(0001₂)을 반환함
 · ② z & ① → z & 1 : z의 값 2(0010₂)와 1(0001₂)의 논리곱(&) 연산의 결과인 0(0000₂)을 반환함
 ④ 조건이 참이면 'z*x'의 값을 저장하고, 아니면 'z/x'의 값을 z에 저장한다. 조건이 참이므로 'z*x'의 값 0을 z

에 저장한다.

x > 5 && z <= 3

①

②

③

- ① x > 5 : x의 값 7은 5보다 크므로 참(True)을 반환
- ② z <= 3 : z의 값 0은 3보다 작으므로 참(True)을 반환
- ③ ① && ② → True && True : 두 항목 모두 참(True)이므로 z에는 'z*x'의 값 0이 저장됨

⑤ z의 값 0을 출력한다.

결과 0

⑥ main() 함수에서의 'return 0;'은 프로그램의 종료를 의미한다.

[문제 18]

super

[해설]

```
class Rectangle {
    int x, y;
    ④ Rectangle(int x, int y) {
    ⑤     this.x = x;
    ⑥     this.y = y;
    }
    int getArea( ) {
        return x*y;
    }
}

class Square extends Rectangle {    // Square를 클래스 정의하고 부모 클래스로 Rectangle을 지정하면
                                     Rectangle에 속한 변수와 메소드를 상속받으므로, Square 클래스
                                     에서는 Rectangle의 변수와 메소드를 사용할 수 있게 된다.

    ② Square(int a) {
    ③     super(a, a);
    }
    ⑧ int getSquareArea( ) {
    ⑨     return x*y;
    }
}

public class Main {
    public static void main(String[] args) {
    ①     Square sq = new Square(10);
    ⑦⑩    System.out.println(sq.getSquareArea( ));
    }
}
```

① Square 클래스의 객체 변수 sq를 선언한다. 10을 인수로 하여 Square 클래스의 생성자를 호출한다.

② Square 클래스의 생성자인 Square() 메소드의 시작점이다. ②번에서 전달한 10을 a가 받는다.

객체 변수 sq
Square
Square(int a)
a
10

③ 부모 클래스의 생성자를 호출하며, 인수로 a와 a, 즉 10과 10을 전달한다.

· **super** : 상속한 부모 클래스를 가리키는 예약어

④ 부모 클래스인 Rectangle 클래스의 생성자 Rectangle()의 시작점이다. ③번에서 전달한 10과 10을 x와 y가 받는다.

객체 변수 sq				
Square	Rectangle			
Square(int a)	상속된 공용 변수		Rectangle(int x, int y)	
a	x	y	x	y
10			10	10

⑤ 'Rectangle.x = x;'와 동일하다. Rectangle.x에 10을 저장한다.

· **this** : 현재 실행중인 메소드가 속한 클래스를 가리키는 예약어

⑥ Rectangle.y에 10을 저장한다.

Rectangle() 생성자가 종료되면 생성자를 호출했던 ③번으로 돌아간 후 Square() 생성자도 종료되므로 Square() 생성자를 호출했던 ①번의 다음 줄인 ⑦번으로 이동한다.

객체 변수 sq				
Square	Rectangle			
Square(int a)	상속된 공용 변수		Rectangle(int x, int y)	
a	x	y	x	y
10	10	10	10	10

⑦ sq.getSquareArea() 메소드를 호출하여 반환받은 값을 출력한다. sq는 Square 클래스의 객체 변수이므로 Square 클래스의 getSquareArea() 메소드를 호출한다. ③번으로 이동한다.

⑧ 정수를 반환하는 getSquareArea() 메소드의 시작점이다.

⑨ Square 클래스는 부모 클래스인 Rectangle 클래스의 변수를 상속 받았으므로, x*y의 값 100을 함수를 호출했던 ⑩번으로 반환한다.

객체 변수 sq				
Square	Rectangle			
Square(int a)	상속된 공용 변수		Rectangle(int x, int y)	
a	x	y	x	y
10	10	10	10	10

⑩ ⑨번으로부터 반환받은 값 100을 출력한다.

결과 100

[문제 19]

187

[해설]

```
#include <stdio.h>
struct Node {
    struct Node* next;    // Node 구조체의 포인터 변수 next를 선언한다.
    unsigned int x;       // 부호없는 정수형 변수 x를 선언한다.
};
```

struct Node	
struct Node* next (4Byte)	unsigned int x (4Byte)
다음 노드의 주소를 저장할 멤버	데이터를 저장할 멤버

```
int main( ) {
    ❶ struct Node t1 = {0, 5u};
    ❷ struct Node t2 = {0, 7u};
    ❸ struct Node t3 = {0, 11u};

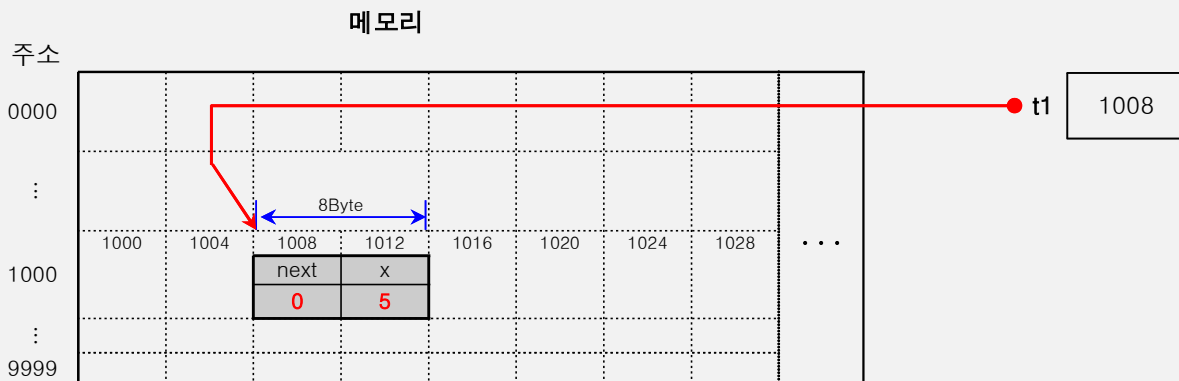
    ❹ t3.next = &t2;
    ❺ t2.next = &t1;

    ❻ struct Node* curr = &t3;
    ❼ int sum = 0;

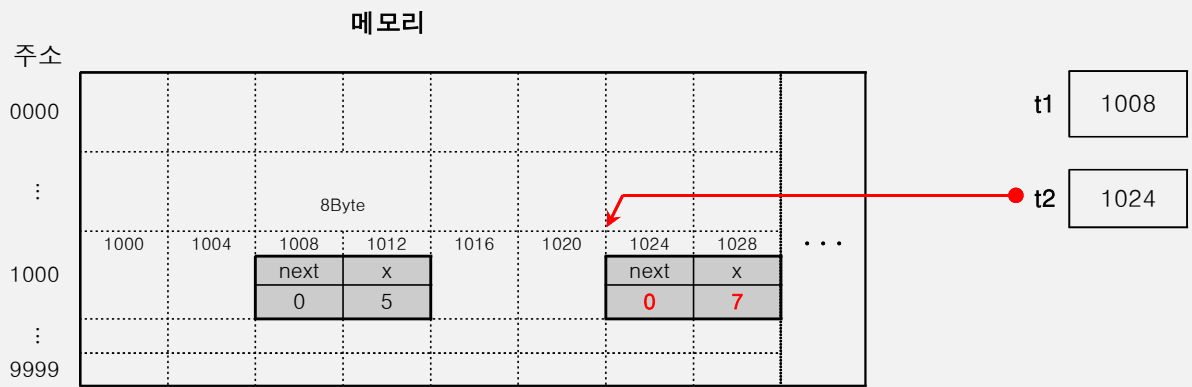
    ❽ while (curr) {
    ❾     sum = sum * 3 + curr->x;
    ❿     curr = curr->next;
    }

    ⓫ sum = (sum ^ 42u) + 100u;
    ⓬ printf("%uWn", sum);
}
```

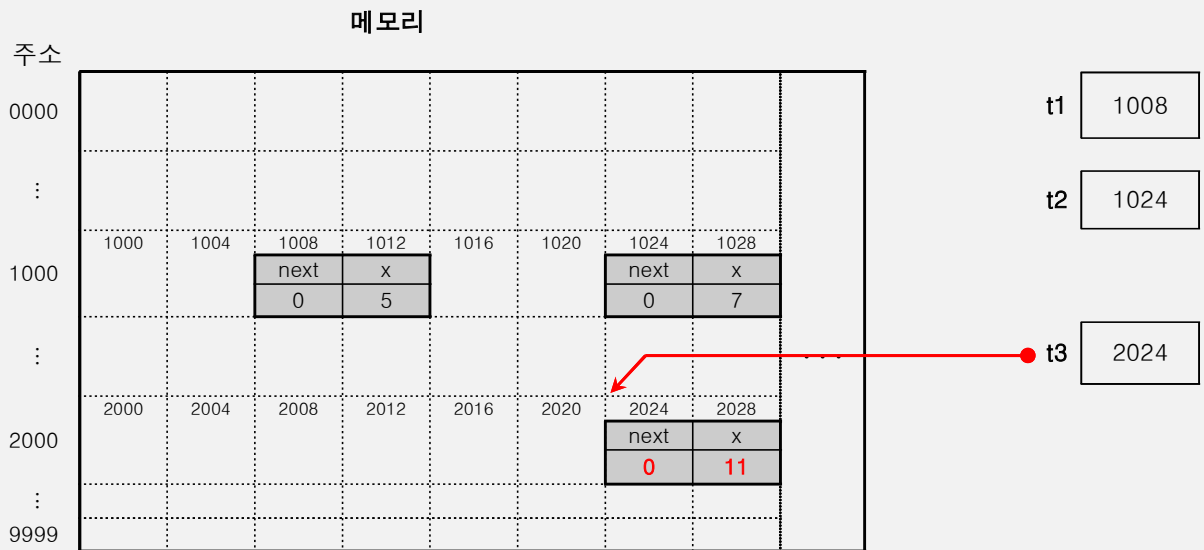
- ❶ Node 자료형 변수 t1을 선언하고, Node의 next에 0을 Node의 x에 부호 없는 정수 5를 저장한다.
 . t1이 가리키고 있는 1008 번지는 Node 구조체의 크기만큼 할당된 공간이므로 t1은 1008번지 이후의 8Byte를 의미한다.
 . 이후 그림에서 지정한 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했다.
 . 5u에서 'u'는 부호 없는 정수형 타입을 의미하는 접미사로 숫자 뒤에 'u'나 'U'를 붙이면 부호 없는 정수형으로 처리된다.
 . next에 0이 저장되어 있는 것은 어떤 곳도 가리키지 않고 있음을 의미한다.



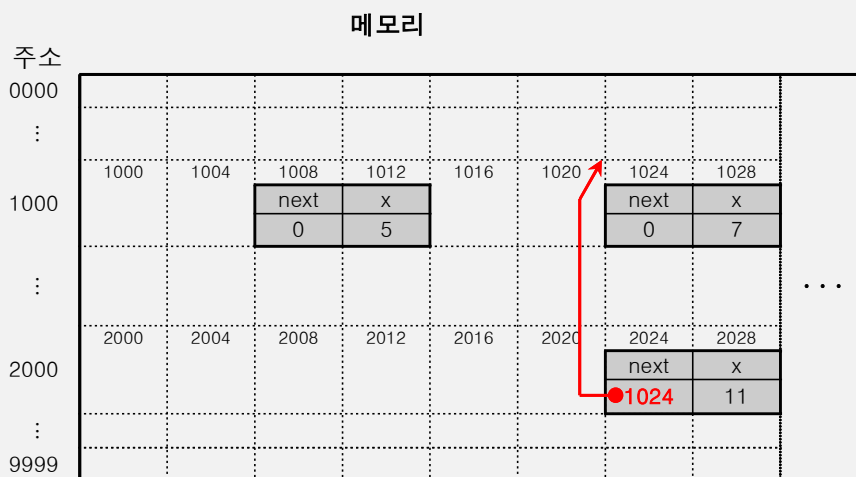
- ❷ Node 자료형 변수 t2를 선언하고, Node의 next에 0을 Node의 x에 7을 저장한다.



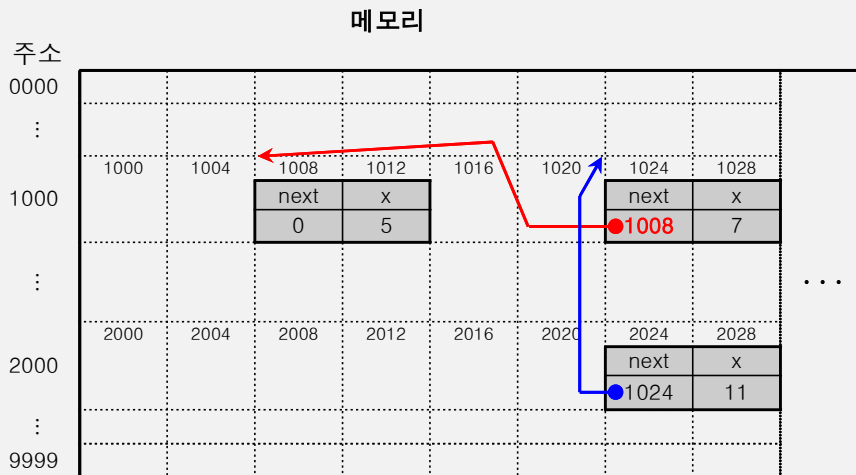
③ Node 자료형 변수 t3을 선언하고, Node의 next에 0을 Node의 x에 11을 저장한다.



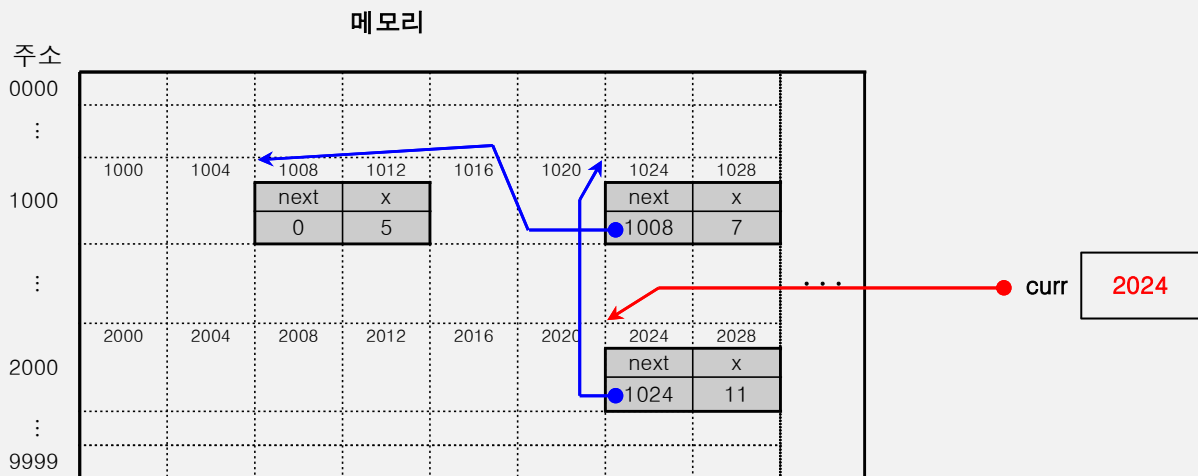
④ t3의 next에 t2의 주소를 저장한다. next는 다음 노드의 주소를 가리키는 것이므로, 이제 t3의 다음 노드는 t2이다.



⑤ t2의 next에 t1의 주소를 저장한다. 이제 t2의 다음 노드는 t1이므로, 노드의 순서는 't3 → t2 → t1'이 된다.



⑥ Node 자료형 포인터 변수 curr을 선언하고, t3의 주소를 저장한다.



⑦ 정수형 변수 sum을 선언하고, 0으로 초기화한다.

⑧ curr이 참인 동안, 즉 0(거짓)이 아닌 동안 ⑨, ⑩번을 반복 수행한다.

⑨ 'sum * 3 + curr->x'의 값을 sum에 저장한다.

⑩ curr이 가리키는 곳의 next를 curr에 저장한다.

반복문 실행에 따른 변수들의 변화는 다음과 같다.

curr->x	sum	curr
	0	2024
11	11	1024
7	40	1008
5	125	0

⑪ '(sum ^ 42) + 100', 즉 sum의 값 125와 42의 비트 XOR(^) 연산의 결과인 87에 100을 더한 값 187을 sum에 저장한다.

$$\begin{array}{r}
 01111101 \quad 125 \\
 \wedge(\text{XOR}) \quad 00101010 \quad 42 \\
 \hline
 01010111 \quad 87
 \end{array}$$

⑫ sum의 값 187을 부호 없는 정수형으로 출력한다.

결과 187

[문제 20]

4

[해설]

SELECT COUNT(COL2)	'COL2'의 개수를 표시한다.
FROM A	<A> 테이블을 대상으로 검색한다.
WHERE COL1 IN (2, 3)	'COL1'이 2 또는 3이거나,
OR COL2 IN (3, 5);	'COL2'가 3 또는 5인 튜플만을 대상으로 한다.

· 질의문의 조건을 만족하는 튜플은 다음과 같습니다.

INDEX	COL1	COL2
1	2	NULL
2	3	6
3	2	3
4	NULL	3
5	4	5

· 모든 행이 조건에 맞으므로, 'COL2' 속성만 추출하면 다음과 같습니다.

COL2
NULL
6
3
3
5

· COUNT(COL2)는 'COL2' 필드의 개수를 계산하지만 'NULL' 값은 제외하므로 COUNT(COL2)의 결과는 4입니다.